Assignment 4
LCR — Left/Center/Right Game

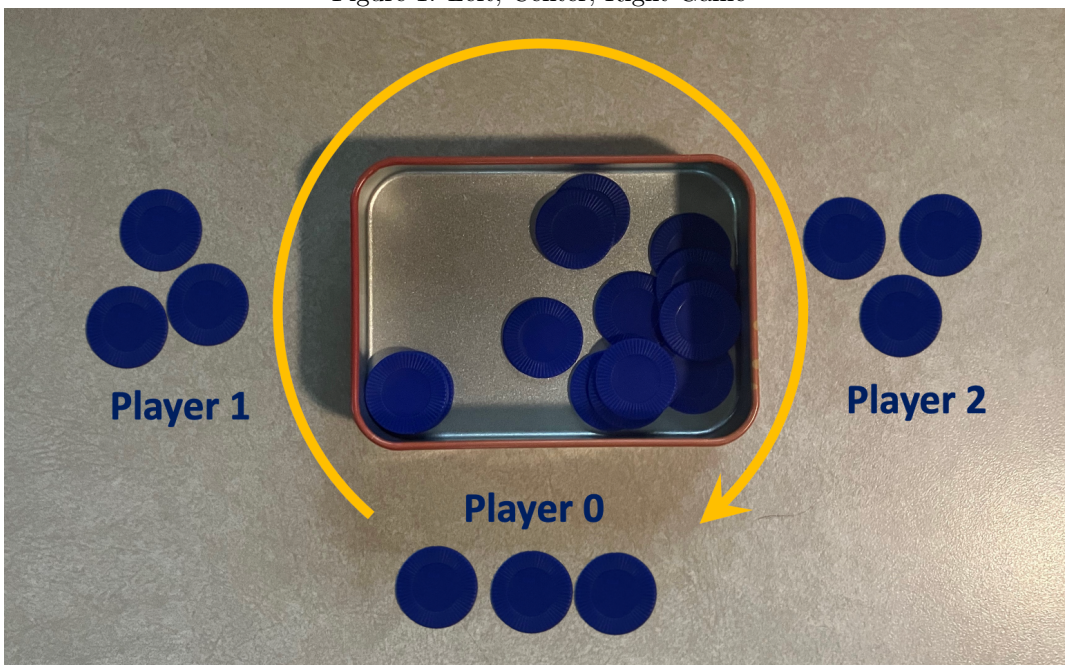
by Jessie Srinivas and Dr. Kerry Veenstra
Based on assignments by Dr. Darrell Long
CSE 13S, Fall 2026
Document version 3

Due November 4th, 2026, at 11:30 pm

1 Introduction

We are going to simulate[1] a simplified version of the dice game Left, Center, Right. (See this video for a version of the rules `https://canvas.ucsc.edu/courses/95951/assignments/906932` .) This game is entirely a game of chance, with no skill or player decisions (except for a die roll). It has relatively simple rules, which makes it easy to implement in C. Fig. 1 shows a game of Left-Center-Right with three players.

Figure 1: Left, Center, Right Game



[1] We are *simulating* it, and not creating it because we don't actually let players play the game, you simply watch it happen.

2 The Rules of the Game

2.1 YouTube Video

The video mentioned above really has the best description of the game that I've seen. It's only 71 seconds long, so take a look. Here's the link again: `https://canvas.ucsc.edu/courses/95951/assignments/906932` .

Now that you've watched the video, get the demo program that we have provided for you:

2.2 Get the Resources

☐ **To do:** In a Terminal, synchronize the **resources** repository on your Github Codespaces instance. Then copy the files for asgn4 into your repository. Remember that the you need to use the `cd` ("change directory") command to set the "current directory" of the Terminal. **The Explorer window does not affect the current directory of the Terminal.**

Note in the first `cd` command below, you want to go to `/workspaces/codespaces-blank` (notice that this "absolute path" starts with a slash `/`), but you can use "auto completion" to reduce the amount of typing that you need to do. That is, instead of typing

```
$ cd /workspaces/codespaces-blank ⏎
```

You can type the first few letters of a directory name followed by a tab key, as demonstrated below.

```
$ cd /wo tab /co tab ⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp resources/asgn4/* yourcruzid/asgn4
$ cd yourcruzid/asgn4
$ ls
```

You should see the three executable files ("reference binaries" or "demo programs") called `lcr_arm`, `lcr_hub`, and `lcr_x86`. You will be running one of these binaries depending on which microprocessor your Terminal is running on. (The first paragraph of the next section explains this.) After you make your own version of the program, you'll have a fourth binary: `lcr`.

2.3 Run the Demo

☐ **To do:** In your repository's `asgn4` directory, run one of the three demo programs that we provided you. Most of you are using Github Codespaces, and so run the command on the left. If you running Ubuntu Linux on a virtual machine, then run one of the other commands.

```
$ ./lcr_hub 3 3
```

```
$ ./lcr_arm 3 3
```

```
$ ./lcr_x86 3 3
```

☐ **To do:** Try a few more examples with **3** players, but try these different random-number seeds instead.

- seed **784**: shortest game (6 dice rolls)
- seed **342**: long game (85 dice rolls)
- seed **958**: short game with a "skipping" line (10 dice rolls)

2.4 The Rules of the Game

Below we describe specifics related to your assignment. You will need to know these specifics when you write the function `run_game()`.

- The user will specify the number players in the game: `num_players`.
- The maximum number of players is `MAX_PLAYERS` (look in the file `lcr_strings.h`).
- Each player will start with 3 chips.
- Players will have names and numbers, with the numbers ranging from `0` to `num_players - 1`.
- The names of the players will be given to you (look in the file `game.c`). You can get the name of player `i` using `player_name[i]`.
- Players will sit in a circle in the order that they appear in the list of names with a pot in the middle of the table (Figure 1).
- The first player in the list of names (player `0`) will go first.
- Each player will roll three dice, unless they have fewer than three chips, and then they will roll one six-sided die for each chip.

  As you saw in the video, the dice of the Left/Center/Right game have special symbols: ●, L, C, and R. However, the easiest way to simulate a die is to use a numbered die that has faces 0 through 5. So we will simulate such a die.

Figure 2: Numbered Dice, 0 – 5.



- If the player rolls a 0, 1, or 2, (we will call these `DOT`) the player will do nothing.
- If the player rolls a 3, (we will call this `LEFT`) the player will pass one chip to the left (the next *higher* player number). Except that the player with number `num_players - 1` passes a chip to player `0`.
- If the player rolls a 4, (we will call this `CENTER`) the player will place one chip in the pot.
- If the player rolls a 5, (we will call this `RIGHT`) the player will pass one chip to the right (the next *lower* player number). Except that the player with number `0` passes a chip to player `num_players - 1`.
- After a player's turn, the next player will take their turn.
- If a player has no chips, they are still in the game, but do not get to roll a die. They can begin rolling again if they are passed a chip by one of their neighbors.
- The game stops when only one player has chips. That player is the winner.

© 2026 Kerry Veenstra

3 Game Abstractions

One of the most important skills to polish as Computer Scientists is the ability to create abstractions for the problems that we need to solve.

3.1 Enumerating the Dice's Symbols

In this game, we differentiate between the "roll" of a die (0 – 5) and the `Symbol`of each roll (`DOT`, `LEFT`, `CENTER`, and `RIGHT`). So, when you roll the die, the top face can have any of the values 0, 1, 2, 3, 4, or 5, but to make decisions about the rolls, you will convert 0, 1, and 2 into `DOT`; convert 3 into `LEFT`; convert 4 into `CENTER`; and convert 5 into `RIGHT`.

To make this simpler in our C program, we are going to do two things. First, we create a new `enum` (enumerated) type. This allows us to create names for the die symbols that we can use in the code.

```
typedef enum { DOT, LEFT, CENTER, RIGHT} Symbol;
```

Let's break that down ...

The first and last words are the `typedef` keyword and the name of the thing that we are defining, which is `Symbol`. This means that we can now use `Symbol` in our code instead of the typing boxed part that we put between `typedef` and `Symbol`. In this case, `Symbol` represents `enum { DOT, LEFT, CENTER, RIGHT}`, meaning that `Symbol` represents an enumerated type that defines 4 names, which are the names of the dice symbols. Internally, these names are just equivalent to numbers (`DOT == 0`, `LEFT == 1`, `CENTER == 2`, `RIGHT == 3`), but using the names is much easier than trying to remember what each of the numbers means.

Putting it all together, we can now create a `Symbol` variable, and use it in standard C syntax. Two examples of how this could be used are as follows:

```
Symbol sym = DOT;

if (sym != LEFT) {
    /* do something */
}
```

The next thing we need to do is create an array that will let us convert from a *roll* (with a value between 0 and 5) into a *symbol* (with values `DOT`, `LEFT`, `CENTER`, and `RIGHT`.) Here is the syntax for that:

```
const Symbol symbol_of_roll[6] = {
    DOT,
    DOT,
    DOT,
    LEFT,
    CENTER,
    RIGHT,
};
```

We repeat `DOT` three times because three of the faces of an LCR die have a ●.

Realize that your die **MUST** be the same as this one for your code to work. (For example, avoid the temptation to alphabetize the symbol names. They must be in this order so that each of the roll values 0 through 5 will get converted into the right symbol.)

A few notes on this array definition:

1. The die array has 6 elements.

2. Each of this array's elements has the type `Symbol`. We can do this because of the enum definition above.

3. This array is defined with the `const` keyword. This means that it cannot be modified after it is created. This is useful to make sure that you don't mess up the die.

4. You can use this array to convert from an integer $0-5$ *roll* value into a *symbol* with a C statement like this:

```
Symbol sym = symbol_of_roll[roll];
```

3.2 Generating Pseudorandom Numbers

```
211327, 653794, 122201, 347348, 516035, 191062, 631229, 511336, 42183, 490570, 99297,
407164, 968971, 475518, 273605, 286032, 331599, 845106, 997353, 215140, 101267, ...
```

We provide you with a pseudorandom number generator (PRNG). We do this because you need to simulate the rolling of dice, and because not all pseudorandom number generators create the same sequence of numbers.

The PRNG that you use will generate values between 0 and 999999. For example, the first three numbers of its default sequence are 211327, 653794, and 122201.

You should use some function to map the PRNG's numbers $(0, 1, 2, ..., 999999)$ into the roll of a die $(0, 1, 2, 3, 4, 5)$. (Hint: we recommend looking at the modulus (`%`) operator in K&R section *2.5 Arithmetic Operators* and section *A7.6 Multiplicative Operators*. The expression `some_random_int % n` will limit the result to integer values between 0 and $n-1$.) If you follow this hint, then the first three die rolls that are implied by the sequence above will be 1, 4, and 5.

Our PRNG is interfaced with the two functions `randi_seed()` and `randi()`. In order to use these functions, you will need to "include" a file in your program using this line:

```
#include "randi.h"
```

The `randi_seed()` function is essential in order to make your program reproducible. Call `randi_seed()` at the beginning of your `play_game()` function. The function call `randi_seed(seed);` sets the random seed of the PRNG to `seed`, which effectively establishes the start point of the pseudorandom number sequence that is generated. This means, after calling `randi_seed(seed);` with a specific value for `seed`, the pseudorandom sequence of numbers that are generated by `randi()` always will appear in the same order.

4 Compiler Warnings Are Good For You

You will be using `clang`'s `-Wall` and `-Werror` command-line options again.

```
clang -c -Wall -Werror file.c        create the "object file" file.o from file.c
```

These command-line options direct the compiler to report common mistakes as errors.

- `-Wall` means "Report all common mistakes as warnings."
- `-Werror` means "Report warnings as errors."

Here are some examples of legal C code that the `-Wall` and `-Werror` command-line options will report as likely errors:

```
for (i = 0; i < 10; ++i) {
    printf("%d\n", i);
    ++i;                        // extra ++i probably is an error
}
```

```
if (x == 1)
    printf("x is 1\n");
    if (x == 2)                 // misleading indentation
        printf("x is 2\n");     // misleading indentation
else
    printf("x is not 2\n");
```

```
if (i == i) {                   // one of the i variables probably is wrong
    ...
}
```

5 Your Task

5.1 Copy the Resource Files

☐ **To do:** Actually, you already did this in Section 2.2.

5.2 Make a Makefile

Why do you use a `Makefile`? Because this is the first assignment in which you have more than one C file. You will be writing `game.c`, but we have given you `lcr.c` (your main) and `randi.c`, and you need to know how to link these three files into a single executable called `lcr`.

Unlike some other programming languages, if you have *three* C program files, you create an executable file through three intermediate "object files" (`lcr.o`, `randi.o`, and `game.o`). The steps are shown below. These are what we want the `make` command to do:

What `make` Does when Compiling Three C Files into One Executable:

```
clang -c -Wall -Werror lcr.c          ⟵ create the "object file" lcr.o from lcr.c
clang -c -Wall -Werror randi.c        ⟵ create the "object file" randi.o from randi.c
clang -c -Wall -Werror game.c         ⟵ create the "object file" game.o from game.c
clang lcr.o game.o randi.o -o lcr     ⟵ create the "executable file" lcr from .o files
```

Whenever you edit `game.c`, `make` will need to repeat the last two lines, which updates `game.o` and then regenerates `lcr`.

☐ **To do:** Create the file **`Makefile`** in the **asgn4** directory. Be certain to use an uppercase **`M`**.

☐ **To do:** For now, **ignore the system's offer to install Makefile Tools**. (We haven't reviewed them.)

☐ **To do:** Review the lecture on October 28th which demonstrated using a `Makefile`.

☐ **To do:** Add text to the Makefile, following the points below:

- Remember that the `make` program will make the **first** target that is listed in your `Makefile`, along with any other dependent files. (The "target" is underlined in the dependency line below.) So if the first dependency line of a `Makefile` is the one below, then the `Makefile` will execute commands to make `lcr` (which is what you want).

When looking at a `Makefile` dependency line, remember, "this *depends on* that ."

When looking at a `Makefile clang` line, remember, "`clang` that *into* this."

```
lcr: lcr.o game.o randi.o
	clang lcr.o game.o randi.o -o lcr
```

- The yellow bar represents a tab character. (Your editor probably doesn't show it like this.) Remember that the commands in a `Makefile` must be indented using an actual "tab" character. As long as you created the **Makefile** with that exact name using the gibhub Explorer window, the editor will enter tab characters correctly.[2] You can check if a character is a tab character by moving the cursor over it. If typing the left and right arrow keys makes the cursor jump left and right by more than one position, then the character is a tab.

- You will need three more pairs of dependency/command lines: Each pair of lines makes one of the `.o` files. A `.o` file depends on its corresponding `.c` file. That is, the name of the `.o` file is to the left of the colon (`:`), and the name of the `.c` file is to the right.

  For example, here is how to make `file.o` from `file.c` ("this depends on that "):

  Notice that the `clang` line is different when compiling a `.c` file into a `.o` file: `clang -c` *file.c* and there is no `-o` option.

```
file.o: file.c $(HEADERS)
	clang -c -Wall -Werror file.c
```

  As was showed in lecture, the `.o` file also depends on the `.h` files that are included in the `.c` file. You can use `HEADERS` and `$(HEADERS)` to simplify things, as was demonstrated.

☐ **To do:** Run your `Makefile` to compile `lcr`. (See the next page.)

Running your `Makefile`

```
$ make          ⟵ That's all that's needed.
```

5.3 Write and Test the Functions

Look in the file `game.c`.

5.3.1 Letter of Symbol

This function accepts a `Symbol` and returns a `char` of the `Symbol` according to this table:

`Symbol`	`char`
`DOT`	`'.'`
`LEFT`	`'L'`
`CENTER`	`'C'`
`RIGHT`	`'R'`

☐ **To do:** Write `letter_of_symbol()`. Consider using a `switch()` statement or a series of `if-then-else` statements.
☐ **To do:** Test `letter_of_symbol()` with a few `assert()` statements in `main()`.

[2] If you are using `vim`, you can insert a tab character using the tab key. If that doesn't work, then in insert mode, use ctrl + V followed by tab.

5.3.2 Min

This function accepts two `int`s and returns whichever one is smallest.
☐ **To do:** Write `min()`.
☐ **To do:** Test `min()` with a few `assert()` statements in `main()`.

5.3.3 Random Roll

This function returns the next roll of a CSE 13S die. That is, it calls randi(), and then converts the result into a number between 0 and 5 using the modulus (`%`) operator.
☐ **To do:** Write `rand_roll()`.
☐ **To do:** Test `rand_roll()` with a few `printf()` statements in `main()`.
☐ **To do:** ***Remove the test from*** `main()` since calling it is not part of the game.

5.3.4 Left Of

This function computes the number of a player that is to the LEFT of another player. Look at Fig. 1 on page 1 to see whether the player to the left has a higher or lower player number.

Remember that the player numbers range from `0` to `num_players - 1`.
☐ **To do:** Write `left_of()`.
☐ **To do:** Test `left_of()` with a few `assert()` statements in `main()`.
☐ **To do:** Be sure that `left_of(0, 3)` returns `1`.
☐ **To do:** Be sure that `left_of(1, 3)` returns `2`.
☐ **To do:** Be sure that `left_of(2, 3)` returns `0`.

5.3.5 Right Of

This function computes the number of a player that is to the RIGHT of another player. Look at Fig. 1 on page 1 to see whether the player to the right has a higher or lower player number.

Remember that the player numbers range from `0` to `num_players - 1`.
☐ **To do:** Write `right_of()`
☐ **To do:** Test `right_of()` with a few `assert()` statements in `main()`.
☐ **To do:** Be sure that `right_of(2, 3)` returns `1`.
☐ **To do:** Be sure that `right_of(1, 3)` returns `0`.
☐ **To do:** Be sure that `right_of(0, 3)` returns `2`.

5.3.6 Print Scores

This function prints all of the players' scores followed by the number of chips in the pot. The scores are in the array `score[]`, which is declared near the top of this file. You can compute the number of chips in the pot (the game has a total of `3 * num_players` chips), or you can maintain the pot's score in a new variable that you declare up where the `score[]` array is.

Here is an example of the output:

```
Current scores:
    3 -- Ada Lovelace
    3 -- Margaret Hamilton
    3 -- Katherine Johnson
    0 -- pot
```

If you want, you can use two format strings in the file `lcr_strings.h`: `CURRENT_SCORES` and `SCORE_ds`.

Look in `lcr_strings.h` to see which format strings are available.

☐ **To do:** Write `print_scores()`.
☐ **To do:** Test `print_scores()` by calling it in `main()`. You can further test it by putting non-zero scores in the `score[]` array.
☐ **To do:** ***Remove the test from*** `main()` since calling it early is not part of the game.

5.4 Document, Write, and Test play_game()

Now write the last function. The file `game.c` has the parameters for `play_game()`, but Section 2 has the details. While rereading the items in Section 2.4, consider these points:

- What needs to happen before the players start rolling the dice (before the main loop)? This is where the pseudo-random number generator is initialized. (Call `randi_seed()`.)
- What actions are implied by the rules? These will become steps in the function. What data are implied by the rules? These will become variable definitions, either in the function or as global variables.

  For example, the rule *"The first player in the list of names (player 0) will go first"* implies that there is an `int` variable that stores the current player number and that the variable is initialized with 0.

  What other program design decisions are implied by the rules?

We are going to grade your comments in the function `play_game()`.

☐ **To do:** Write a function-header comment for the function `play_game()`. Remember to include these parts:

- **Purpose**: What is the function's purpose?
- **Parameters**: What are the names, types, and uses of the functions input parameters? (Or if appropriate, mention that there are none.)
- **Returns**: What is the type and purpose of the function's return value? (Or if appropriate, mention that there is none.)

☐ **To do:** Also, write some comments in the body of the function, documenting your ideas on the algorithm that you will write.

☐ **To do:** Write `play_game()`. If you change your mind about anything that you had documented, update the function-header comment to match.

© 2026 Kerry Veenstra

5.5 Test Your Program

To receive full credit, the output of your program must match the output of a reference program that we provide. You can use the `diff` program as you did in the Hangman assignment to compare the output of the reference program and the output of your program.

Here are three tests.

Testing in the Terminal

```
$ ./lcr_hub 3 784 > expected.txt  ⟵ Create the expected output for 3 players and a seed of 784.
$ ./lcr 3 784 > got.txt           ⟵ Get your program's output for 3 players and a seed of 784.
$ diff -us expected.txt got.txt   ⟵ Compare the results.
```

Testing in the Terminal

```
$ ./lcr_hub 3 8 > expected.txt    ⟵ Create the expected output for 3 players and a seed of 8.
$ ./lcr 3 8 > got.txt             ⟵ Get your program's output for 3 players and a seed of 8.
$ diff -us expected.txt got.txt   ⟵ Compare the results.
```

Testing in the Terminal

```
$ ./lcr_hub 100 8 > expected.txt  ⟵ Create the expected output for 100 players and a seed of 8.
$ ./lcr 100 8 > got.txt           ⟵ Get your program's output for 100 players and a seed of 8.
$ diff -us expected.txt got.txt   ⟵ Compare the results.
```

5.6 Submit: add/commit/push/submit commit ID

You need to submit these files:

- `Makefile`
- `lcr.c`
- `game.c`
- `randi.c`
- `game.h`
- `randi.h`
- `lcr_strings.h`

☐ **To do:** In the Terminal type:

```
cd yourcruzid/asgn4        (if your Terminal's working directory is not yet your class directory)
git add Makefile *.c *.h
```

Remember that the tab key will complete a filename, and so instead of `Makefile` you can type `Ma`tab .
Do **not** type `git add .` because doing so will add the `.o` files and the executable file, which you never should submit.

☐ **To do:** `git commit -m '`*the changes that you made*`'` (you can give any reason for this commit)

☐ **To do:** `git push`

☐ **To do:** Run the `asgn4` pipeline at **git.ucsc.edu**.

☐ **To do:** Submit your final commit ID to Canvas by November 4th at 11:30 pm.

© 2026 Kerry Veenstra

6 Revisions

Version 3 Clarified submission instructions.

Version 2 Fixed link to video. Corrected an instance of `asgn3` to `asgn4`. Added date of **Makefile** lecture.

Version 1 Original.

© 2026 Kerry Veenstra